RAG that sees charts, connects documents, and stops ignoring half your data

Attendee guide. Build four kinds of retrieval-augmented generation over ten messy files: text RAG done right, multimodal RAG for pictures, scans, audio and video, graph RAG, and an agent that uses all three.

Speaker: Kiruthika Subramani · DevFest Troy, NY 2026

Built with Gemini on Google Cloud and the Google Agent Development Kit

Written for complete beginners: it starts from an empty laptop

Every screenshot and terminal block in this guide comes from a real run.

Long terminal output is shortened

Contents

1 The problem, and what you will build

2 Before you start

3 Install three programs

4 Set up Google Cloud

5 Get the code and check your setup

6 Meet the ten files

7 Step 1: what a program sees in a file

8 Step 2: chunks and embeddings

9 Step 3: naive RAG, and where it goes wrong

10 Step 4: text RAG done right

11 Step 5: multimodal RAG

12 Step 6: graph RAG

13 Step 7: agentic RAG

14 The app: five answers side by side

15 The scorecard: which RAG for which question

16 How the code is put together

17 Test every part

18 When something goes wrong

19 Small glossary

1 The problem, and what you will build

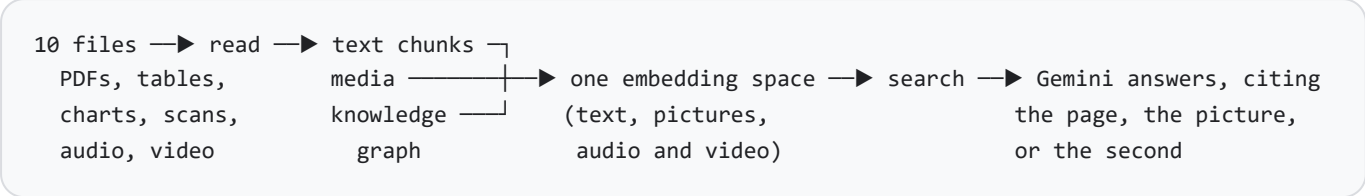
Your RAG reads text. That is it. Someone uploads a PDF with a chart, and the answer is right there in the picture, but the pipeline acts as if it does not exist.

RAG, retrieval-augmented generation, means: before a model answers, find the passages that hold the answer and give them to it. The model then answers from your documents instead of from memory, and can say where each fact came from.

The tutorial version of RAG breaks in three ways that every real company hits:

IT BREAKS WHEN	EXAMPLE IN THIS WORKSHOP	THE FIX YOU BUILD
Two versions of a document disagree	The 2024 refund policy says 30 minutes, the 2026 one says 60	Text RAG with metadata filters and reranking (step 4)
The answer is in a picture, a scan, a recording or a video	The busiest month exists only as a bar in a chart	Multimodal RAG (step 5)
The answer is spread over several files	Incident, station, manager and phone number live in four files	Graph RAG (step 6)

Then, in step 7, you hand all three techniques to an agent and let it choose. You build every one on the same ten files and test every one on the same nine questions, so you see exactly where each one wins and where it breaks.



Text RAG: the right passage, the right version | Multimodal: charts, scans, audio, video | Graph: facts across files

2 Before you start

You do not need to have built RAG before, or to have used Google Cloud. This guide starts from an empty laptop.

YOU WILL NEED	DETAILS
A laptop	Windows, macOS or Linux, with permission to install programs
A Google account	Any Gmail address works
A bank card	Google Cloud asks for one to open a billing account, even for small use
About 40 minutes	For sections 3 to 5, which you only do once
About an hour	For the seven steps

What it costs. Each question is a few Gemini calls: a fraction of a cent. The agent makes several calls per question. Rebuilding the index from scratch, which you do not have to do, costs a few cents. The full scorecard, 45 answers, costs well under a dollar.

2.1 How to open a terminal

ON	DO THIS
Windows	Press the Windows key, type **PowerShell**, press Enter
macOS	Press Cmd+Space, type **Terminal**, press Enter
Linux	Press Ctrl+Alt+T

2.2 How to read the dark boxes

Dark boxes show a terminal. A bright line after the green **>** is something **you type**, followed by Enter. Do not type the **>** itself. Grey lines are what the computer answers. Answers written by Gemini will be worded a little differently on your machine; the facts should be the same.

```
> python --version
Python 3.13.0
```

3 Install three programs

Python runs the code. Git downloads it. The Google Cloud CLI signs you in. If you already have one, run its check and skip ahead.

3.1 Python

1. Go to **python.org/downloads** and press the yellow **Download Python** button.
2. **Windows:** on the first screen of the installer, tick **Add python.exe to PATH**, then press **Install Now**.
3. **macOS:** press Continue through the installer.
4. Close your terminal and open a new one.

```
> python --version
Python 3.13.0
```

Any version from 3.11 up works. On macOS and Linux the command may be `python3`.

3.2 Git

Windows: download the installer from **git-scm.com/downloads** and press Next on every screen. **macOS:** type `git --version`; if Git is missing, a window offers to install it. Then open a new terminal.

```
> git --version
git version 2.54.0.windows.1
```

3.3 The Google Cloud CLI

Go to **cloud.google.com/sdk/docs/install**, choose your system and follow the installer. On Windows, accept the defaults. Then open a new terminal.

```
> gcloud --version
Google Cloud SDK 571.0.0
```

4 Set up Google Cloud

Gemini runs on Google Cloud. You need a project, a billing account, one service switched on, and your laptop signed in.

4.1 Create a project

1. Open **console.cloud.google.com/projectcreate** and sign in.
2. Type a name, for example **rag-workshop**. Write down the **Project ID** shown under it, like `rag-workshop-123456`.
3. Press **Create**.

4.2 Connect a billing account

Open **console.cloud.google.com/billing**. Create a billing account if you have none (new accounts are usually offered free credit). Then on **console.cloud.google.com/billing/projects**, find your project and choose **Change billing** if it has none.

4.3 Sign in, and point gcloud at your project

```
> gcloud auth login
You are now logged in as [you@gmail.com].
```

```
> gcloud config set project YOUR_PROJECT_ID
Updated property [core/project].
```

4.4 Switch on Vertex AI

Gemini, and the embedding model this workshop uses, are reached through **Vertex AI**. Every service is off until you switch it on:

```
> gcloud services enable aiplatform.googleapis.com
Operation "operations/acf.p2-..." finished successfully.
```

4.5 Sign in the programs on your laptop

```
> gcloud auth application-default login
Credentials saved to file: [...application_default_credentials.json]
```

This is a second, different sign-in: 4.3 signed in *you*, for the `gcloud` command; this one signs in *programs* on your laptop, which is why the code holds no API key. **Pick the same account as in 4.3.** If your browser shows two Google accounts, choosing the other one is the most common setup problem (section 18).

```
> gcloud auth application-default set-quota-project YOUR_PROJECT_ID
Credentials saved to file: [...application_default_credentials.json]
```

5 Get the code and check your setup

5.1 Download the code

```
> cd Documents
> git clone https://github.com/Kiruthika162003/rag-workshop.git
> cd rag-workshop
```

Everything from here on is typed inside the `rag-workshop` folder.

5.2 A private Python environment

```
WINDOWS (POWERSHELL)
> python -m venv .venv
> .venv\Scripts\activate
```

```
MACOS AND LINUX
> python3 -m venv .venv
> source .venv/bin/activate
```

The prompt now starts with `(.venv)`. Switch it on again in every new terminal.

> **PowerShell says running scripts is disabled?** Run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, answer Y, and try again.

5.3 Install the packages

```
> pip install -r requirements.txt
```

It ends with a long line that starts `Successfully installed`.

This also installs the workshop's own code as a package called `core`, so every step in `learn/` can use it. It takes two or three minutes.

5.4 Tell the code which project is yours

```
WINDOWS: copy    ·    MACOS AND LINUX: cp
> copy .env.example .env
```

Open `.env` in any text editor and replace `your-project-id` with your Project ID:

```
GOOGLE_CLOUD_PROJECT=rag-workshop-123456
GOOGLE_CLOUD_LOCATION=global
GOOGLE_GENAI_USE_VERTEXAI=TRUE
```

5.5 Check everything

```
> python check.py

Checking your setup

  [ ok ]  Python 3.13
  [ ok ]  The packages are installed
  [ ok ]  The settings file points at project rag-workshop-123456
  [ ok ]  gcloud is installed
  [ ok ]  Programs on this computer are signed in
  [ ok ]  Gemini answered: ready   (gemini-3.8-flash)
  [ ok ]  Embeddings work: 768 numbers per passage   (gemini-embedding-2)
  [ ok ]  The index is built

Everything works. Start with: python learn/1_read_the_files.py
```

A `[ FIX ]` line names the problem, and the line under it says what to type.

> **The index is already built.** The repository ships with it, so you can start straight away. Section 16 shows how to rebuild it from the files, which takes about five minutes.

6 Meet the ten files

Collar City Bikes is a made-up bike share in Troy, New York. Its files are messy on purpose, the way real company files are. They live in `data/docs`.

FILE	KIND	DATED	WHAT MAKES IT HARD
refund-policy-2024.pdf	Policy	2024-02-01	Out of date: says 30 minutes and $1,200
refund-policy-2026.pdf	Policy	2026-01-15	Replaces the 2024 one: 60 minutes and $950
2025-annual-ridership-report.pdf	Report	2026-03-10	Monthly rides, the station table and the truck route are pictures
winter-hours-flyer.pdf	Flyer	2025-11-20	A scan: the page is one picture, with no text at all
incident-report-IR-2026-031.pdf	Incident	2026-03-12	Names the station, not the person who signs off
station-operations-memo-2026.pdf	Memo	2026-01-05	A table of station managers, one of them out of date
board-minutes-2026-04-22.pdf	Minutes	2026-04-22	Changes who manages Monument Square, from 1 May
staff-directory-2026.pdf	Directory	2026-02-01	The phone numbers, in a table
rider-voicemail-2026-04-06.wav	Voicemail	2026-04-06	A complaint that exists only as audio
dock-training-video.mp4	Training	2026-02-10	The dock reset steps exist only on screen in a video

2025 Annual Ridership Report

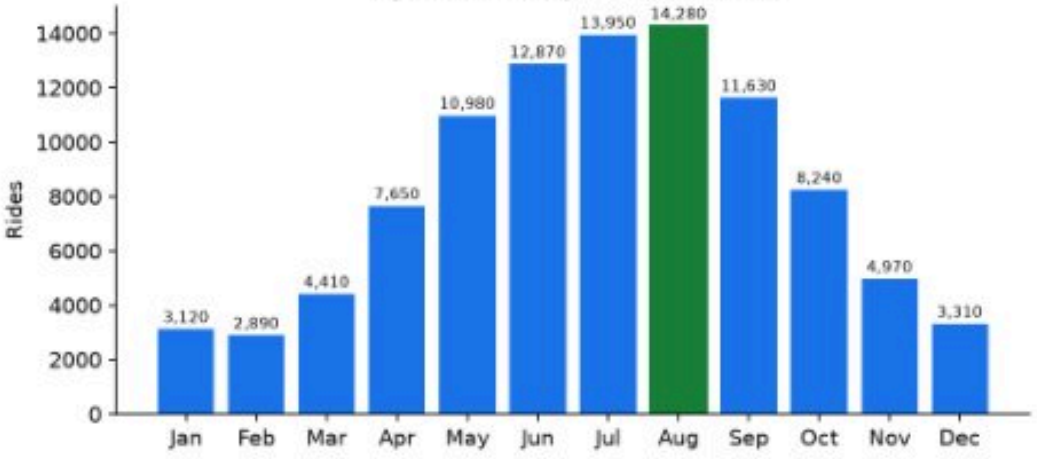
Published 10 March 2026 · Planning team

Summary

2025 was our busiest year so far. Riders took close to one hundred thousand trips, up from 71,000 in 2024. The summer months carried the year: see Figure 1 for the month by month numbers.

Growth came mostly from students and from commuters crossing the river. The two stations near campus and downtown were the busiest, as Table 1 shows. Lansingburgh grew the fastest from a small base.

Figure 1. Rides per month, 2025



Stations

We ran 22 stations at the end of 2025. Table 1 lists the six busiest, with the number of docks at each.

Table 1. Our six busiest stations

Station	Docks	Rides in 2025
Monument Square	24	21,400
RPI Student Union	30	19,800
Riverfront Park	18	16,950
Troy Atrium	16	11,200
Green Island Bridge	14	9,300
Lansingburgh	12	7,450

FIG 1 The annual report. The text says "see Figure 1 for the month by month numbers". The numbers are only in the chart, and the station table is a pasted picture.

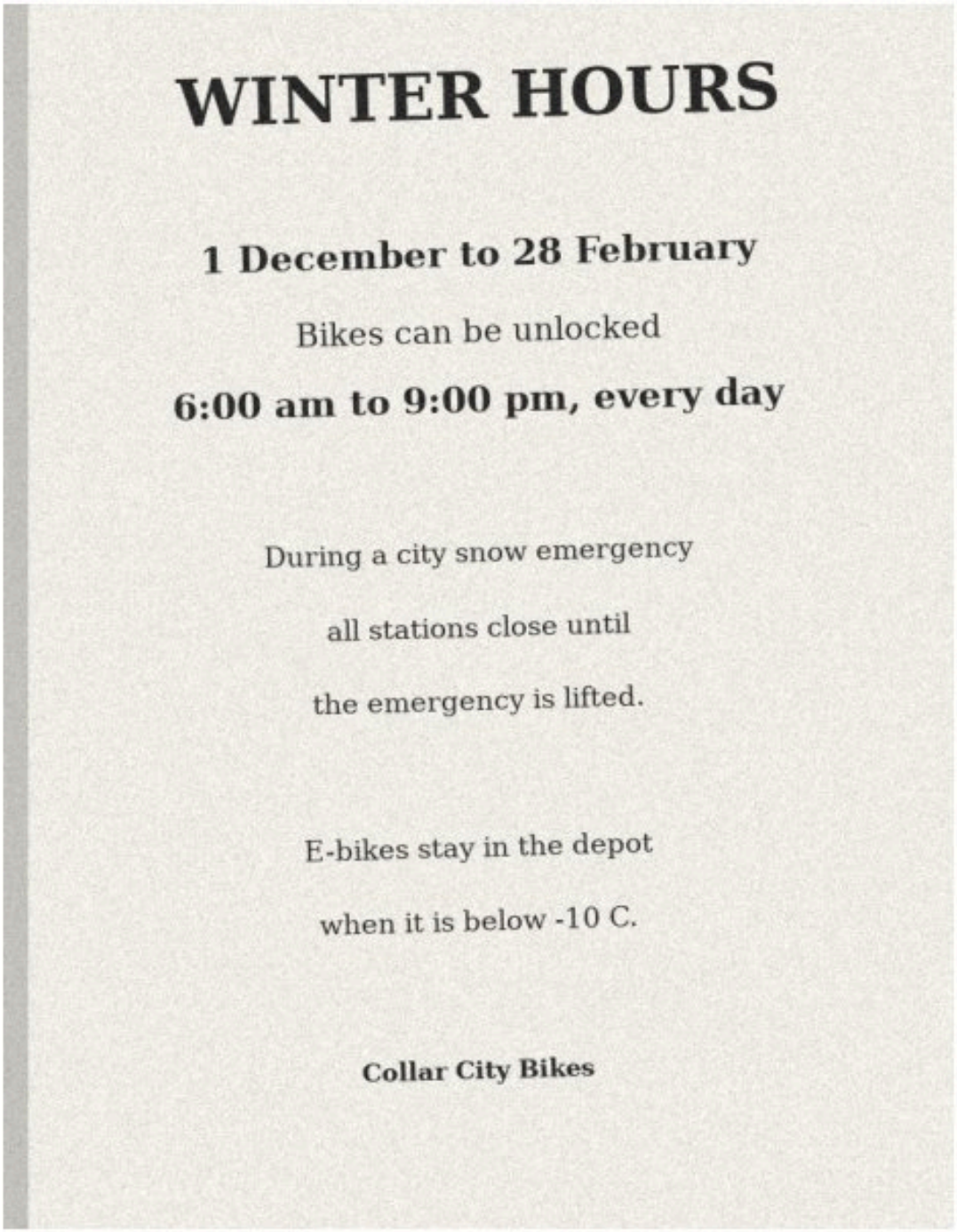


FIG 2 The winter hours flyer, scanned slightly crooked. To a text reader this page is empty.

The nine test questions in `core/questions.py` are built on these files. Each one needs something a simple pipeline lacks:

QUESTION	NEEDS	CORRECT ANSWER
How long do I have to report a broken bike to get a refund?	the newest version	60 minutes
What is the charge for a lost bike?	the newest version	$950
Which month had the most rides in 2025, and how many?	a chart	August, 14,280
Which station has the most docks?	a table that is a picture	RPI Student Union, 30
In what order does the rebalancing truck visit the stations?	a diagram	Monument Square, RPI Student Union, Lansingburgh, Riverfront Park
What are the winter opening hours?	a scanned page	6:00 am to 9:00 pm
Which bike lost power on the Green Island Bridge, and who reported it?	audio	Bike 9012, Sam Carter
How do I free a bike that is stuck in a dock?	video	Hold the red reset button for 5 seconds
The dock from incident IR-2026-031 has just been repaired. Who has to sign it off, and what is their phone number?	facts from four files	Daniel Kim, 518-555-0158

7 Step 1: what a program sees in a file

Before any AI, there is reading. Most RAG pipelines ask a PDF for its text layer, and anything without one quietly disappears.

TEXT LAYER The characters a PDF stores as text. A scanned page, a pasted screenshot or a chart has none.

PIECES What a careful reader produces: sentences, table rows, and pictures, audio and video kept as they are.

METADATA Facts about a file rather than in it: its title, kind, date and series. Step 4 filters on it.

```
> python learn/1_read_the_files.py

file                              kind       date         naive reader gets   careful reader
finds
2025-annual-ridership-report.pdf  Report     2026-03-10         1108 chars   {'heading': 6,
'sentence': 13, 'picture': 3}
dock-training-video.mp4           Training   2026-02-10            0 chars   {'video': 1}
rider-voicemail-2026-04-06.wav    Voicemail  2026-04-06            0 chars   {'audio': 1}
station-operations-memo-2026.pdf  Memo       2026-01-05          638 chars   {'heading': 2,
'sentence': 5, 'row': 6}
winter-hours-flyer.pdf            Flyer      2025-11-20           48 chars   {'picture': 1}

The scanned winter flyer, as a naive reader sees it:
    Collar City Bikes · Flyer · internal Page 1 of 1

A table row, read as one sentence that keeps its column names:
    Station: Monument Square; Station manager: Priya Raman

Two versions of one policy share a series, so we can tell which is newer:
    2024-02-01  Refund Policy (February 2024)
    2026-01-15  Refund Policy (January 2026)
```

Three things to notice:

1. **The flyer gives 48 characters**, and they are the header and the footer. The audio and the video give nothing. To a text-only pipeline, a third of these files say nothing useful.
2. **Table rows keep their column names.** Read cell by cell, "Monument Square" and "Priya Raman" become two unrelated words. Read as a row, they stay one fact.
3. **Every file has metadata**: a kind, a date and a series. "Refund Policy (February 2024)" and "Refund Policy (January 2026)" share the series *Refund Policy*, so the code can tell which one is newer.

The reader is `core/documents.py`. It also drops the header and footer repeated on every page, so they cannot pollute every chunk, and keeps pieces in reading order, top to bottom.

8 Step 2: chunks and embeddings

Two ideas every RAG is built on. A chunk is the piece of a document that gets retrieved. An embedding turns a chunk into numbers, so pieces with similar meaning end up close.

8.1 Chunks

FIXED CHUNKS	Every 500 characters, wherever that lands, often in the middle of a word.
SEMANTIC CHUNKS	Sentences stay together while they talk about the same thing. A new chunk starts at a heading, at a page break, or where the topic changes.
CHUNK HEADER	Each chunk starts with its document title, date and section, so "within 60 minutes" knows it comes from the 2026 policy.

```
> python learn/2_chunks_and_embeddings.py

Fixed chunks, every 150 characters, cut wherever they land:
   | Refund Policy Version 2 · In force from 15 January 2026 · ... Faulty bikes Report a f |
   | aulty bike in the app within 60 minutes of unlocking it. We refund the full ride ... |

Semantic chunks of the same page, from the index. Each one starts with where it came from:
   | Refund Policy (January 2026) (2026-01-15) · Faulty bikes / Report a faulty bike in the app
within 60 minutes o
   | Refund Policy (January 2026) (2026-01-15) · Memberships / Annual memberships cost $95. They can
be refunded wi
   | Refund Policy (January 2026) (2026-01-15) · Lost or stolen bikes / A bike that is not returned
within 24 hours
```

"Report a f | aulty bike" is what fixed chunking does to a sentence. Semantic chunks keep each policy section whole, and a chunk never crosses a page, so every chunk has exactly one page number to cite.

8.2 Embeddings and similarity

An embedding is a list of 768 numbers. Similar meanings point in similar directions, and the **cosine similarity** between two embeddings says how close they are: 1.0 is the same meaning.

```
How close is each sentence to the question "How long do I have to report a broken bike?"?
    0.875  Report a faulty bike in the app within 60 minutes of unlocking it.
    0.543  Annual memberships cost $95.
    0.517  The board approved four stations in Albany.
```

The question shares almost no words with the right sentence ("broken" against "faulty"), and it still scores highest. That is what embeddings buy you over keyword search.

8.3 One space for text, pictures, audio and video

This workshop uses `gemini-embedding-2`, which embeds a sentence, a chart, a voicemail and a video into the same space. So a question typed in words can find a picture:

```
The same space holds pictures, audio and video. Which file matches "rides per month in 2025"?
    0.570  audio    rider-voicemail-2026-04-06.wav
    0.514  picture  2025-annual-ridership-report.pdf
    0.451  picture  2025-annual-ridership-report.pdf

The voicemail beat the chart. Scores between a sentence and a recording run high: the modality gap.
Scoring each file as itself and by its description, the better of the two, fixes the order:
    0.740  picture  2025-annual-ridership-report.pdf
    0.614  picture  2025-annual-ridership-report.pdf
    0.595  audio    rider-voicemail-2026-04-06.wav
```

The modality gap. A sentence compared with a recording is not on the same scale as a sentence compared with a sentence: audio scored high against everything. The fix used here, and in production, is to store each picture, recording and video twice: as itself, and by a description or transcript Gemini wrote when the index was built. Search takes the better of the two scores.

9 Step 3: naive RAG, and where it goes wrong

The version in most tutorials: chop the text into 500-character pieces, find the three most similar to the question, paste them into a prompt.

These three lines are all of it:

```
found = Store("naive").search(embed(question), k=3)
context = "\n\n".join(chunk["text"] for chunk in found)
reply = ask(NAIVE_PROMPT.format(context=context, question=question))
```

```
> python learn/3_naive_rag.py

Question: The dock from incident IR-2026-031 has just been repaired. Who has to sign it off, and
what is their phone number?

The three most similar chunks:
    0.763  incident-report-IR-2026-031.pdf, page 1
    0.722  station-operations-memo-2026.pdf, page 1
    0.696  staff-directory-2026.pdf, page 1

Answer:
    Based on the context, the dock from incident IR-2026-031 is at the Monument Square station.
    Repaired docks must be signed off by the station manager before returning to service.

    * Person to sign it off: Priya Raman (Station Manager for Monument Square)
    * Phone number: 518-555-0142
```

It sounds right, and it is wrong. Priya Raman managed Monument Square in January. The board minutes, which naive RAG never retrieved, gave the station to Daniel Kim from 1 May. Nothing told it which file is newer, and nothing let it follow a fact from one file to the next.

Try it on a question that lives in a picture:

```
> python learn/3_naive_rag.py "Which month had the most rides in 2025, and how many?"
Answer:
    Based on the provided context, this information is not available. The report states that
    the "summer months carried the year" and directs readers to Figure 1 for month-by-month
    numbers, but Figure 1 and the specific monthly breakdown are not included in the text.
```

It even knows Figure 1 exists. It just cannot see it.

10 Step 4: text RAG done right

Four moves turn the tutorial pipeline into one you can trust with text: retrieve wide, filter by metadata, rerank, and cite.

RETRIEVE	Twelve candidates by similarity, from semantic chunks with headers.
FILTER	By metadata, before ranking: kind of document, date, newest version only. A filtered-out chunk can never win.
RERANK	Gemini reads each candidate and scores 0 to 10 whether it answers the question. Similar words are not enough. Weak candidates are dropped.
CITE	A numbered source after every fact, with its page. Today's date is in the prompt, so "most recent" means something.

```
> python learn/4_text_rag.py

Question: How long do I have to report a broken bike to get a refund?

1-2. Retrieved and filtered: 12 candidates, old versions removed
    0.842  2026-01-15  Refund Policy (January 2026), page 1
    0.758  2026-01-15  Refund Policy (January 2026), page 1
    0.754  2026-01-15  Refund Policy (January 2026), page 1
    0.728  2026-03-12  Incident report IR-2026-031, page 1
    0.713  2026-03-12  Incident report IR-2026-031, page 1

3. Reranked: up to four kept, any that score under 3 dropped:
    10/10  Refund Policy (January 2026):  Report a faulty bike in the app within 60 minutes of
unlocking it. We refund the...

4. Answer:
    You must report a faulty bike in the app within 60 minutes of unlocking it to receive a refund
[1].
    The full cost of the ride will then be refunded to your original payment card within 5 business
    days, along with a $5 ride credit [1].

Sources:
    [1] Refund Policy (January 2026), page 1   file:///.../data/docs/refund-policy-2026.pdf#page=1

Source [1], with the passage highlighted: out/source-1.png
```

Refund Policy

Version 2 · In force from 15 January 2026 · Customer care · This version replaces the policy of 1 February 2024.

Faulty bikes

Report a faulty bike in the app within 60 minutes of unlocking it. We refund the full ride to the original payment card within 5 business days, and add a $5 ride credit to say sorry.

Memberships

Annual memberships cost $95. They can be refunded within 14 days of purchase if no rides were taken.

Lost or stolen bikes

A bike that is not returned within 24 hours is treated as lost. The member is charged $950, down from $1,200, because most lost bikes are now found by their GPS tracker.

FIG 3 out/source-1.png: the page the answer came from, with the passage highlighted. The link in the terminal opens the PDF at that page.

10.1 Metadata filters and the most recent content

Three filters are built into the store (`core/store.py`). Try each one:

```
> python learn/4_text_rag.py "What is the charge for a lost bike?" --all-versions

1-2. Retrieved and filtered: 12 candidates
    0.842  2024-02-01  Refund Policy (February 2024), page 1
    0.826  2026-01-15  Refund Policy (January 2026), page 1

3. Reranked: up to four kept, any that score under 3 dropped:
    10/10  Refund Policy (January 2026):  A bike that is not returned within 24 hours is treated as
lost...
     9/10  Refund Policy (February 2024):  A bike that is not returned within 24 hours is treated as
lost...

4. Answer:
    The charge for a lost bike is $950 [1]. An earlier policy set the fee at $1,200, but that
    source is out of date [1, 2].
```

With `--all-versions` the 2024 policy comes back first: it is the most similar. The reranker scores both versions highly, because both answer the question. The answer is still right, because every source carries its date, today's date is in the prompt, and the rules say to trust the most recent. Without the flag, the 2024 policy never reaches the prompt at all. **Filtering is stronger than asking.**

```
> python learn/4_text_rag.py "What did the board decide?" --kind Minutes --after 2026-04-01

1-2. Retrieved and filtered: 3 candidates, old versions removed
    0.731  2026-04-22  Board meeting minutes, 22 April 2026, page 1
    0.698  2026-04-22  Board meeting minutes, 22 April 2026, page 1
    0.693  2026-04-22  Board meeting minutes, 22 April 2026, page 1

4. Answer:
    The board approved opening four stations in Albany in September 2026 with a $310,000 budget [1].
    They appointed Priya Raman as Head of Operations and assigned Daniel Kim as station manager for
    Monument Square in addition to Troy Atrium, both effective 1 May 2026 [2]. Additionally, sixty
    e-bikes were ordered for June delivery [3].
```

Only three chunks pass both filters, and all three are from the April minutes.

In production, metadata filters do two more jobs. They enforce permissions (only the documents this user may see), and they keep answers fresh (skip anything older than a date, or anything replaced by a newer version).

11 Step 5: multimodal RAG

The question searches pictures, scans, audio and video by meaning, and what matches goes to Gemini as it is. A chart goes in as an image, a voicemail as audio.

Two indexes, one question: text chunks are reranked as in step 4, and media is searched in the same embedding space. Both go into the prompt:

```
passages = rerank(question, find_text(question, latest_only=True), keep=3)
media = find_media(question)
answer(question, passages + media)
```

```
> python learn/5_multimodal_rag.py

Question: Which month had the most rides in 2025, and how many?

Media that match, found by meaning:
    0.781  picture  2025-annual-ridership-report.pdf:  This is a bar chart showing the number of
rides per month in the year 2025...
    0.661  picture  2025-annual-ridership-report.pdf:  This is an image of a data table listing
bike-share station statistics...

Answer:
    The month with the most rides in 2025 was August [2]. It had 14,280 rides [2].

The page the picture is on, with the picture outlined: out/picture.png
```

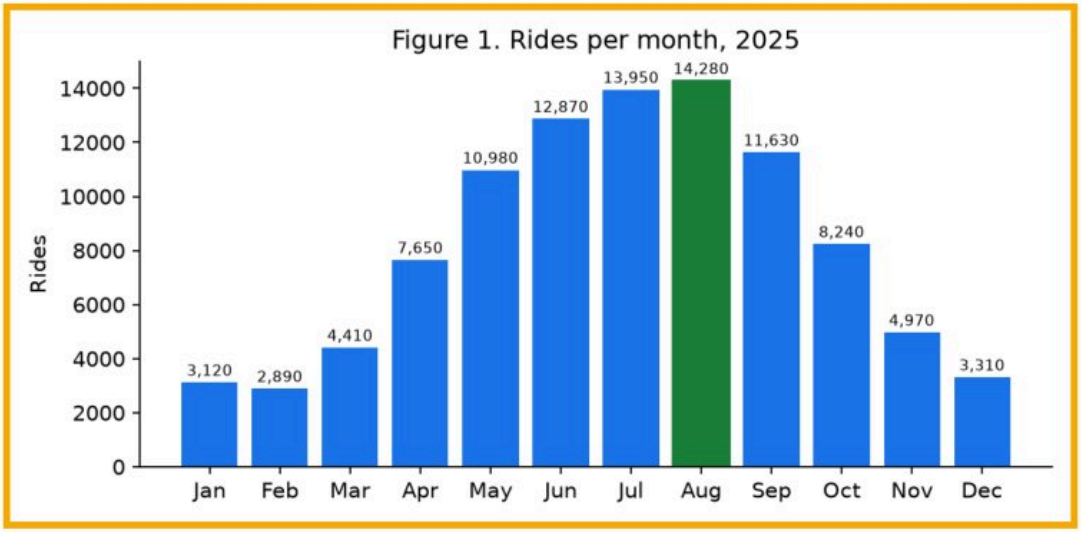
2025 Annual Ridership Report

Published 10 March 2026 · Planning team

Summary

2025 was our busiest year so far. Riders took close to one hundred thousand trips, up from 71,000 in 2024. The summer months carried the year: see Figure 1 for the month by month numbers.

Growth came mostly from students and from commuters crossing the river. The two stations near campus and downtown were the busiest, as Table 1 shows. Lansingburgh grew the fastest from a small base.



Stations

We ran 22 stations at the end of 2025. Table 1 lists the six busiest, with the number of docks at each.

Table 1. Our six busiest stations

Station	Docks	Rides in 2025
Monument Square	24	21,400
RPI Student Union	30	19,800
Riverfront Park	18	16,950
Troy Atrium	16	11,200
Green Island Bridge	14	9,300
Lansingburgh	12	7,450

FIG 4 out/picture.png: the chart the answer was read from, outlined on its page.

Now audio and video:

```
> python learn/5_multimodal_rag.py "Which bike lost power on the Green Island Bridge, and who
reported it?"
Answer:
    Bike number 9012 lost power halfway across the Green Island Bridge [2, 0:10, 0:16].
    The incident was reported by Sam Carter [2, 0:00].
```

```
> python learn/5_multimodal_rag.py "How do I free a bike that is stuck in a dock?"
Media that match, found by meaning:
    0.766  video    dock-training-video.mp4:  [0:00] A slide appears with the text "Dock
training"...
    0.688  audio    rider-voicemail-2026-04-06.wav:  [0:00] Hi, this is Sam Carter, member number
20417...

Answer:
    To free a bike that is stuck in a dock, hold the red reset button on the dock for 5 seconds
    [2, 0:06]. Next, wait up to 30 seconds for the dock light to turn green [2, 0:12]. If the
    light is still red, tag the dock "OUT OF SERVICE" and call the technician on duty [2, 0:18].
```

Citations for recordings and videos carry the **time**. In the app (section 14), clicking one plays the clip from that second.

If sending the original fails. A request that carries a video or a recording is the slowest kind there is, and the service can time out. When that happens the answer step tries again from the media's transcript or description (`core/answer.py`): the answer is still grounded and still cites the time.

Where do the times come from? When the index was built, Gemini listened to the voicemail and watched the video, and wrote a transcript and a description with a time on every line, like `[0:07] On Saturday the e-bike I took from Riverfront Park lost all power`. The original file still goes to Gemini at answer time; the transcript is what lets a citation point at a moment.

12 Step 6: graph RAG

Some answers live in no single file. A name in one, a date in another, a phone number in a third. Graph RAG stores facts as links and walks across them.

FACT	A link: subject, relation, object, and a date when it starts. *Daniel Kim → station manager of → Monument Square, since 2026-05-01.*
BUILDING	When the index was built, Gemini read every chunk, transcript and description and wrote down its facts: 152 of them. Each points back to its chunk, so it can still be cited.
WAY IN	The names in the question, plus the names in the two chunks a vector search finds. "The dock that failed" names nothing; the search finds the incident report.
WALK	Every fact within three links: incident, to station, to its manager, to the phone number.

```
> python learn/6_graph_rag.py --node "Monument Square"
    Daniel Kim  --station manager of-->  Monument Square   (since 2026-05-01)
    sixty e-bikes  --will be placed first at-->  Monument Square
    Incident report IR-2026-031  --happened at-->  Monument Square   (since 2026-03-12)
    Priya Raman  --manages-->  Monument Square
    Monument Square  --has docks-->  24
    Monument Square  --rides in 2025-->  21,400
    morning loop  --stop 1-->  Monument Square
```

Two facts about who manages Monument Square, from two documents, on one node. One has a date. That is the conflict naive RAG could not see.

```
> python learn/6_graph_rag.py

Where the walk starts: dock, incident report ir-2026-031

The walk found 40 facts. The path the answer follows, both ends named in it:
    [4] Daniel Kim  --station manager of-->  Monument Square   (since 2026-05-01)
    [6] Priya Raman  --manages-->  Monument Square
    [9] Daniel Kim  --role-->  Station manager
    [9] Daniel Kim  --has phone-->  518-555-0158

Answer:
    The dock from incident IR-2026-031 is located at Monument Square [1], and a repaired dock can
    only return to service after the station manager signs off the repair [3]. While Priya Raman
    was previously listed as station manager [6], that memo is out of date compared to the April
    2026 board meeting minutes, which state that Daniel Kim took over as station manager of
    Monument Square on 1 May 2026 [4]. Therefore, Daniel Kim must sign it off [3, 4], and his
    phone number is 518-555-0158 [9].
```

Four files, one answer, and it explains why the memo is out of date. The walk also uses the same metadata filter as text RAG: a fact read from a replaced document, like the 2024 policy, is dropped before the answer is written.

Graph RAG is not free. Building the graph is one Gemini call per chunk, and the graph is only as good as the facts it read: relations come out worded differently ("manages", "station manager of"). Production systems normalise them, and many store the graph in a graph database such as Spanner Graph or Neo4j. Here it is one JSON file, so you can open it: `index/graph.json`.

13 Step 7: agentic RAG

Hand the techniques to an agent as tools, and let it decide which to use, with which filters, in which order.

The agent is in `core/agent.py`, built with the Google Agent Development Kit. It has four tools, each a plain Python function with a docstring the model reads:

TOOL	WHAT IT DOES
`search_documents(query, kind, after, newest_version_only)`	Text RAG from step 4. The agent chooses the filters itself
`search_media(query)`	Finds pictures, scans, recordings and videos, with their descriptions
`look_at(ref, question)`	Sends one picture or clip to Gemini, for a detail the description missed
`search_graph(question)`	Walks the knowledge graph from step 6

```
> python learn/7_agentic_rag.py

What the agent decided to do:
    search_documents(query='IR-2026-031')
    search_graph(question='Who signs off dock repairs for IR-2026-031 and what is their phone
number?')
    search_documents(query='Daniel Kim')

Answer:
    Incident IR-2026-031 occurred at dock 7 of the Monument Square station [t7]. Under company
    policy, a repaired dock goes back into service only after the station manager signs off the
    repair [t9]. While Priya Raman originally managed Monument Square [t19], Daniel Kim took over
    as station manager for Monument Square starting May 1, 2026 [t5]. Daniel Kim's direct phone
    number is 518-555-0158 [t17].

Sources it cited:
    [t7] text     Incident report IR-2026-031, page 1
    [t9] text     Incident report IR-2026-031, page 1
    [t19] text     Station operations memo, page 1
    [t5] text     Board meeting minutes, 22 April 2026, page 1
    [t17] text     Staff directory, page 1
```

Nobody told it to use the graph. It searched, walked the graph, then checked the answer it found with a second search for Daniel Kim. Its citations are the ids of chunks (`t`) and media (`m`). Run it twice and the tool calls may differ: the agent decides each time.

Try a question that needs a scan:

```
> python learn/7_agentic_rag.py "What are the winter opening hours, and what happens in a snow emergency?"
```

The price of an agent. It got all nine test questions right, and it is the slowest: several model calls per question, 15 to 60 seconds each. Use one when questions vary and you cannot know in advance which retrieval they need.

14 The app: five answers side by side

```
> python app/app.py
 * Running on http://127.0.0.1:5050
```

Open **http://localhost:5050**. Type a question or click one of the chips: each chip says what the question needs. The question goes to all five kinds at once, and each card fills in when its answer is back, with its time and a ✓ or ✗ against the expected answer.

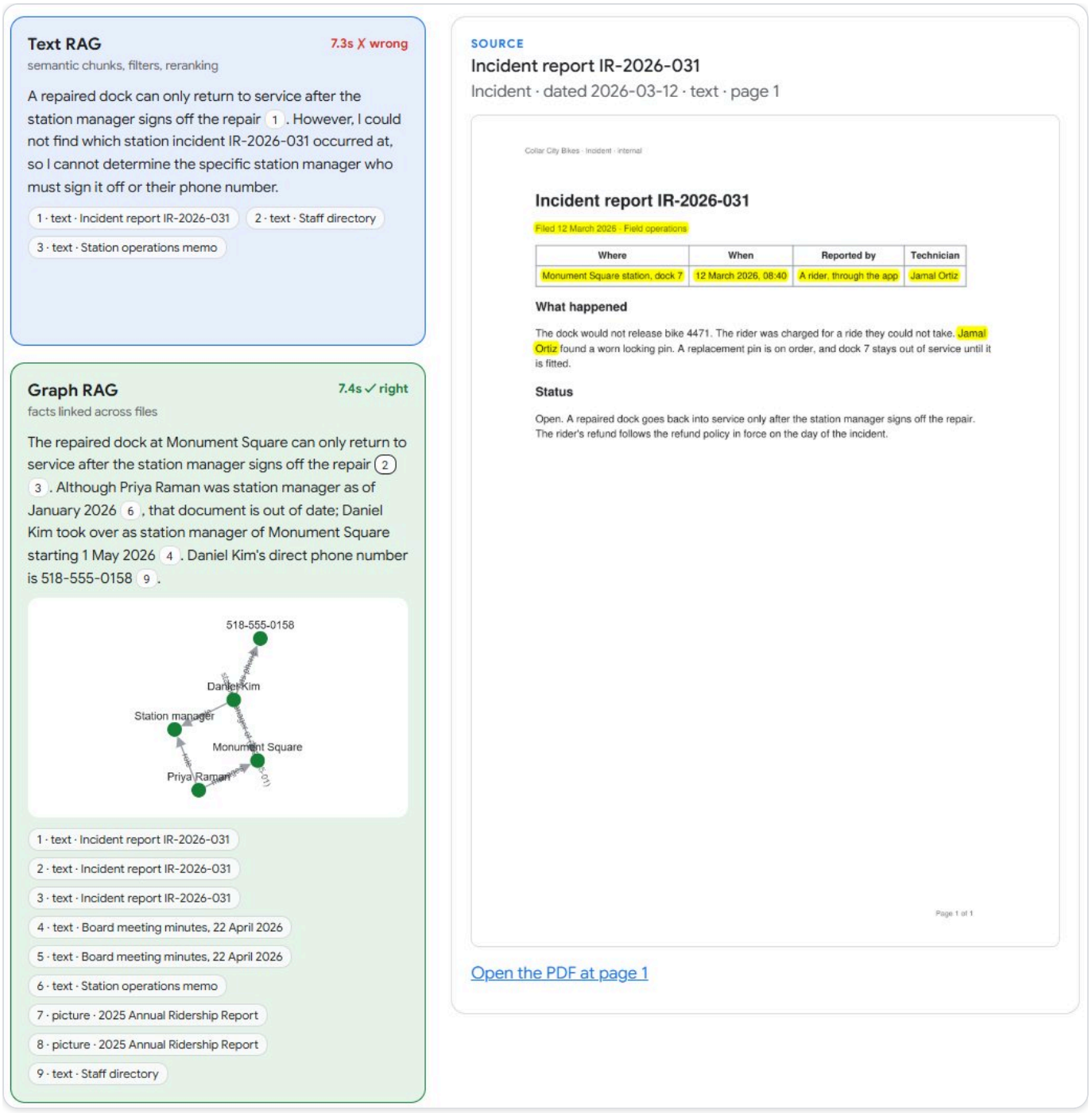


FIG 5 Part of the screen for the four-file question. Text RAG says it cannot connect the files. Graph RAG finds Daniel Kim and draws the path it walked. On the right, the incident report a citation points to, with the passage highlighted. Naive RAG, above, gave the out-of-date manager; the agent, below, found Daniel Kim too.

Click any citation chip to open its source on the right. Each card shows the sources its answer cites; the ones it retrieved but did not use are folded under *retrieved but not cited*.

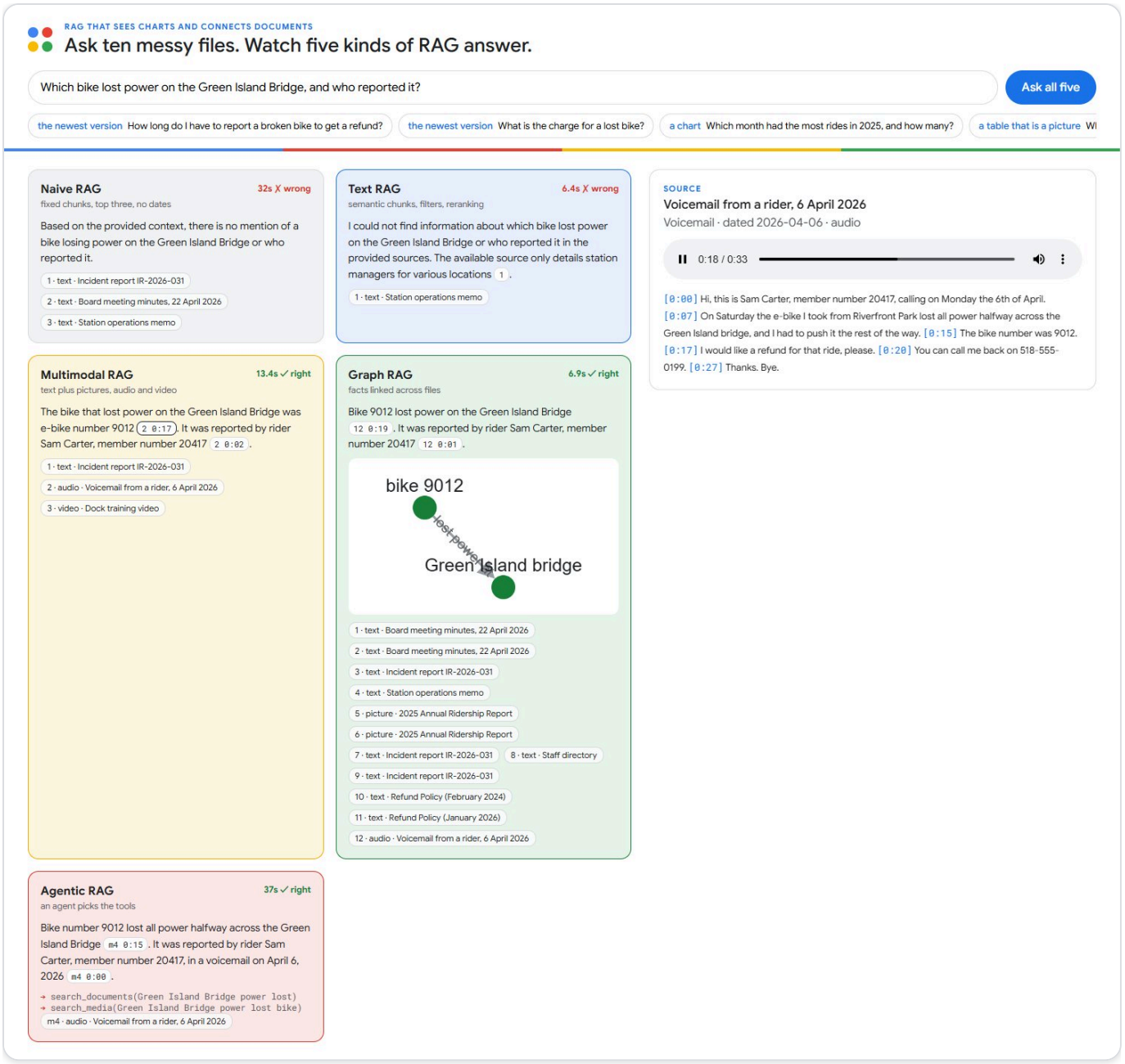


FIG 6 A citation to the voicemail plays it from the cited second. Every time in the transcript can be clicked.

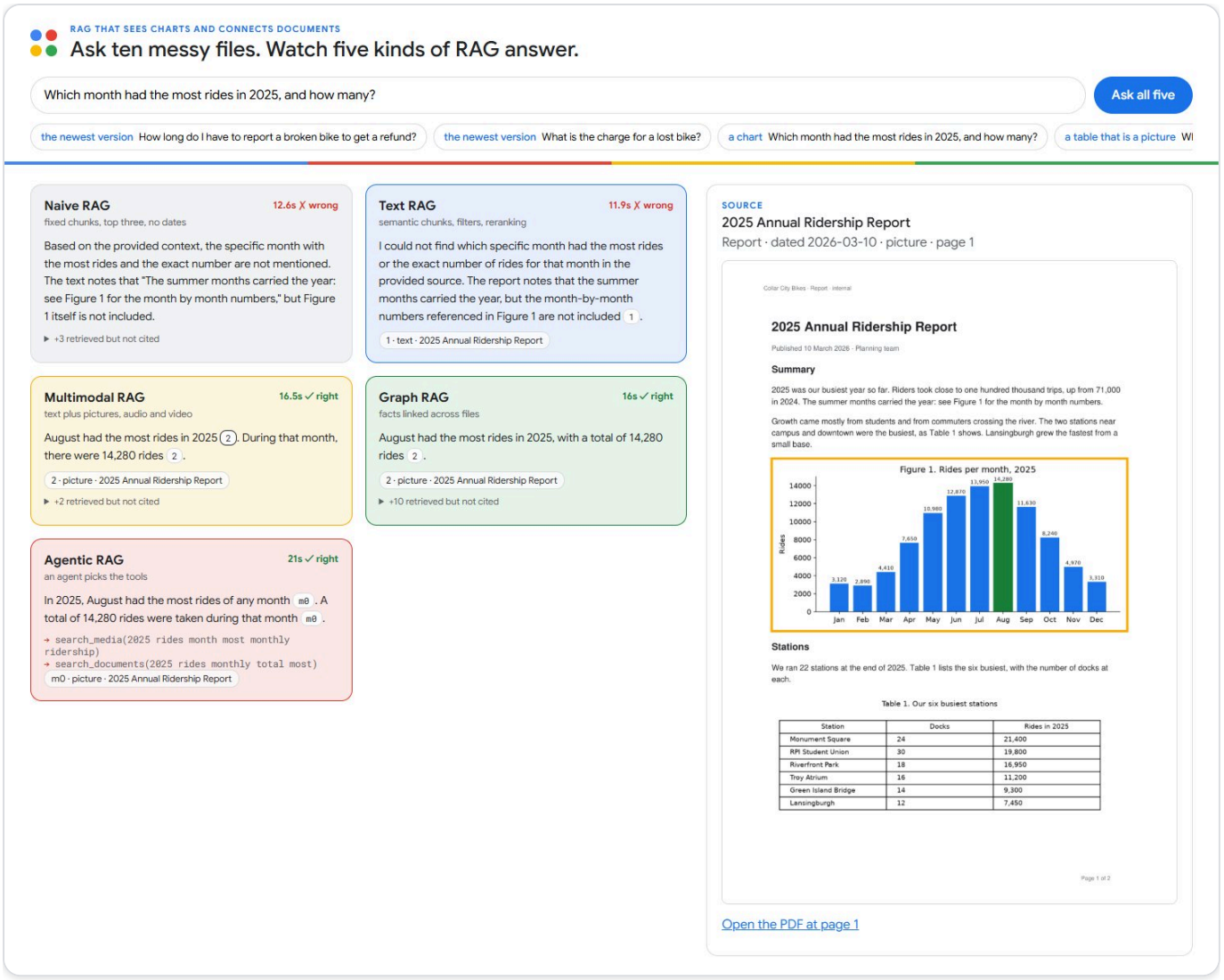


FIG 7 The chart question: the text-only kinds say the numbers are not there; the others read August, 14,280 off the chart.

Stop the app with **Ctrl+C**.

15 The scorecard: which RAG for which question

```
> python evaluate.py
Every answer is in index/scorecard.json
```

QUESTION	NEEDS	NAIVE	TEXT	MULTIMODAL	GRAPH	AGENT
refund	the newest version	✓	✓	✓	·	✓
lost-bike	the newest version	✓	✓	✓	✓	✓
busiest-month	a chart	·	·	✓	✓	✓
most-docks	a table that is a picture	·	·	✓	·	✓
truck	a diagram	·	·	✓	✓	✓
winter	a scanned page	·	·	✓	·	✓
voicemail	audio	·	·	✓	✓	✓
stuck-dock	video	·	·	✓	·	✓
sign-off	facts from four files	·	·	·	✓	✓
right		**2 / 9**	**2 / 9**	**8 / 9**	**5 / 9**	**9 / 9**

Naive RAG's two ticks are luck, not design: both refund policies happened to land in its top three, it saw both dates in the text, and it listed both versions. Ask it something where the two versions are not retrieved together and nothing protects it. Graph RAG's misses are honest: it says it cannot find answers that live in a picture, a scan or a video, because that is not what it is for.

Each answer is marked right when it contains the expected facts, in order where order matters, and does not say it gave up. Every answer is saved in `index/scorecard.json`. Gemini words answers differently each run, so a score can move by one between runs.

USE	WHEN	WATCH OUT FOR
Text RAG	Your answers are in text: policies, manuals, tickets. The right default	Old versions: filter on dates and series. Weak matches: rerank
Multimodal RAG	Answers live in charts, tables, scans, photos, recordings or video	The modality gap: score media by description too. Large files: split long video and audio into segments
Graph RAG	Answers join facts across files: people, places, dates, relations	The cost of building the graph, and relations worded inconsistently
Agentic RAG	Questions vary and you cannot know which retrieval they need	More calls, more time, more to evaluate

Use the simplest one your questions need, and keep a scorecard like this one so you know when it stops being enough.

16 How the code is put together

FILE	WHAT IT DOES
`data/make_documents.py`	Writes the ten files. Each PDF is a small piece of HTML
`core/documents.py`	Reads every file into pieces, with page and position
`core/chunking.py`	Fixed and semantic chunking
`core/gemini.py`	The only file that calls Google: `embed()` and `ask()`
`core/store.py`	A vector store in numpy and JSON, with the metadata filter
`core/graph.py`	Reads facts, stores them, walks them
`core/rag.py`	Naive, text, multimodal and graph RAG, built from small functions
`core/agent.py`	The agent and its four tools
`core/answer.py`	Builds the prompt with numbered, dated sources and returns a cited answer
`build_index.py`	Reads every file once and writes everything in `index/`
`evaluate.py`, `tests/`	The scorecard, and fast checks with no model calls

16.1 Rebuild the index

```
> python build_index.py
10 files
naive   16 fixed-size chunks
text    21 semantic chunks
media   6 pictures, recordings and videos, each embedded as itself and by its description
graph   152 facts
```

It takes about five minutes: most of it is Gemini describing the media and reading facts for the graph. To change the files, edit `data/make_documents.py`, run it, then rebuild.

16.2 Taking it to production

- **Vector store:** the numpy store does what Vertex AI Vector Search, AlloyDB or pgvector do at scale. Keep the metadata next to the vectors and filter before ranking.
- **Reranking:** one Gemini call is clear for teaching; the Vertex AI ranking API does the same job faster and cheaper at volume.
- **Managed RAG:** Vertex AI RAG Engine chunks, embeds and retrieves for you, when you do not need to control each step.

- **Freshness:** rebuild the index when files change, and keep dates and series in the metadata so the newest version wins.
- **Evaluation:** keep a question set with expected answers, like `core/questions.py`, and run it after every change.

17 Test every part

TEST	RUN	YOU SHOULD SEE
Setup	`python check.py`	Eight lines of `[ ok ]`
Fast checks, no model calls	`python -m pytest`	`9 passed`
Reading	`python learn/1_read_the_files.py`	48 chars for the flyer, 0 for the audio and video
Embeddings	`python learn/2_chunks_and_embeddings.py`	The chart first once descriptions count
Naive RAG	`python learn/3_naive_rag.py`	Priya Raman: out of date
Newest version	`python learn/4_text_rag.py`	60 minutes, and `out/source-1.png`
Filters	`python learn/4_text_rag.py "What did the board decide?" --kind Minutes --after 2026-04-01`	Only the April minutes
A chart	`python learn/5_multimodal_rag.py`	August, 14,280
Audio	`python learn/5_multimodal_rag.py "Which bike lost power on the Green Island Bridge, and who reported it?"`	Bike 9012, Sam Carter, with a time
Video	`python learn/5_multimodal_rag.py "How do I free a bike that is stuck in a dock?"`	Red reset button, 5 seconds
Graph	`python learn/6_graph_rag.py`	Daniel Kim, 518-555-0158
Agent	`python learn/7_agentic_rag.py`	Its tool calls, then Daniel Kim
App	`python app/app.py`, open localhost:5050	Five cards; a citation opens a highlighted page or plays audio
Everything	`python evaluate.py`	The scorecard in section 15

18 When something goes wrong

WHAT YOU SEE	WHAT IT MEANS	WHAT TO DO
`No module named core` or another package	The virtual environment is off, or the install did not finish	Activate it (5.2), then `pip install -r requirements.txt`
`403 PERMISSION_DENIED` on `aiplatform.endpoints.predict`	Programs are signed in as a different Google account than the one that owns the project	Run `gcloud auth application-default login` again and pick the right account
"has not been used in project" or "is disabled"	Vertex AI is off	`gcloud services enable aiplatform.googleapis.com`
"billing" in the error	The project has no billing account	Section 4.2
`No API key was provided`	`.env` is missing `GOOGLE_GENAI_USE_VERTEXAI=TRUE`	Copy `.env.example` again and set your project
`429 RESOURCE_EXHAUSTED` or very slow answers	The project's per-minute Gemini quota is used up, often after running the scorecard	Wait a minute or two; every call already retries six times with growing pauses
`504 DEADLINE_EXCEEDED`	The service timed out, usually on a video or recording	Ask again. The answer step already falls back to the transcript once
"The index has not been built yet"	`index/` is missing	`python build_index.py`
Port 5050 already in use	The app is already running in another terminal	Stop it with Ctrl+C there
An answer is worded differently from this guide	Gemini writes each answer fresh	Nothing to fix; check the facts

19 Small glossary

WORD	MEANING
RAG	Retrieval-augmented generation: find the passages first, then let the model answer from them
Chunk	The piece of a document that is stored and retrieved
Semantic chunking	Cutting where the topic changes instead of every N characters
Embedding	A list of numbers for a piece of text, a picture, audio or video; similar meanings sit close together
Cosine similarity	How close two embeddings are; 1.0 is the same meaning
Vector store	Where embeddings are kept and searched
Metadata	Facts about a file: title, kind, date, version
Metadata filter	Keeping only chunks whose metadata passes, before ranking
Reranking	Reading the candidates again and ordering them by whether they answer the question
Citation	A pointer from a fact in the answer to the page, picture or moment it came from
Multimodal	Working with more than text: pictures, audio, video
Modality gap	Similarity scores across kinds of media are not on the same scale
Knowledge graph	Facts stored as links between named things
Hop	One step along a link in the graph
Agent	A model that decides which tools to call, and when it has enough to answer
Grounded	An answer built only from retrieved sources, never from memory